Machine Learning — Unit 3 Exam Answers

Unit 3: Ensemble Learning (Syllabus Unit V)

SPPU B.E. (AI & DS) 2019 Pattern, Course 417521 · Exam: 70 marks, 2.5 hours · Paper unit = question pair (Q1/Q2 → Unit 1, Q3/Q4 → Unit 2, Q5/Q6 → Unit 3, Q7/Q8 → Unit 4).

Each answer is written for **(X + 2) marks**, where X = highest marks ever asked. Sources cite your PDFs by file name and PDF page number.

1. Unit Snapshot

No.	Question	Max Marks	Years Asked	Importance
U3-1	What is ensemble learning? Explain its need, advantages and limitations, and any two ensemble techniques.	9	Nov-Dec 2023, Nov-Dec 2024, Nov-Dec 2025	★★★
U3-2	Explain the voting ensemble and the different types of voting mechanisms.	9	Nov-Dec 2023, Nov-Dec 2024, Nov-Dec 2025	★★★
U3-3	Explain the stacking approach of ensemble learning with example. What methods are used for variance reduction in stacking?	9	Nov-Dec 2024, Nov-Dec 2025	★★★
U3-4	Explain the Adaptive Boosting (AdaBoost) algorithm in detail.	6	Nov-Dec 2023, Nov-Dec 2025	★★
U3-5	Explain the Random Forest ensemble method with an example.	6	Nov-Dec 2023, Nov-Dec 2025	★★
U3-6	Differentiate between Homogeneous and Heterogeneous ensemble methods, with examples.	6	Nov-Dec 2023, Nov-Dec 2025	★★
U3-7	Explain Gradient Boosting ensemble learning technique.	9	Nov-Dec 2024	★
U3-8	Differentiate the Bagging and Boosting approaches of ensemble learning.	6	Nov-Dec 2023	★

2. Must-Do First (★★★)

1. **U3-1** — What is ensemble learning? Explain its need, advantages and limitations, and any two ensemble techniques. (9 M)
2. **U3-2** — Explain the voting ensemble and the different types of voting mechanisms. (9 M)
3. **U3-3** — Explain the stacking approach of ensemble learning with example. What methods are used for variance reduction in stacking? (9 M)

3. Full Answers

U3-1. What is ensemble learning? Explain its need, advantages and limitations, and any two ensemble techniques.

Importance: ★★★ | **Max asked:** 9 M → **write for 11 M** | **Asked in:** Nov-Dec 2023, Nov-Dec 2024, Nov-Dec 2025

Source: ML Unit 3-6.pdf, pp. 68–71 (Sec. 5.1 Introduction, 5.1.1 Need of Ensemble Learning, advantages and limitations; book pp. 5-1 to 5-4) and pp. 75–77 (bagging, boosting).

Asked as: *What is the ensemble learning? Explain any two ensemble learning techniques.* (Nov-Dec 2023 Q6(b), 6 M) | *Explain Following with respect to Ensemble learning. i) Need of Ensemble Learning ii) Advantages of Ensemble methods iii) Ensemble learning limitations* (Nov-Dec 2024 Q5(a), 9 M) | *Explain the concept of ensemble learning. Why is ensemble learning considered beneficial in machine learning?* (Nov-Dec 2025 Q5(a), 6 M)

Definition

Ensemble learning combines the predictions of **multiple base models (weak learners)** to produce one **stronger, more accurate and more stable** model. Idea: "**wisdom of the crowd**" – many models together make fewer errors than any single model.

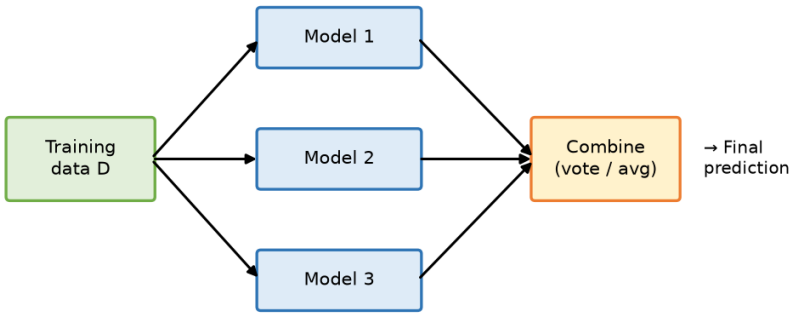


Fig. General ensemble: several models trained on data, outputs combined

- **Training data D:** given to every base model (same or resampled).
- **Model 1…n:** base learners (same or different algorithms).
- **Combine:** voting (classification) or averaging (regression) → final prediction.

1. Need of Ensemble Learning (why it is beneficial)

1. **Reduce variance** – single deep trees overfit; averaging many reduces it (bagging).
2. **Reduce bias** – weak learners combined sequentially become strong (boosting).
3. **Improve accuracy & generalisation** – errors of different models cancel out.
4. **Robustness / stability** – less sensitive to noise and to a particular training sample.
5. **Handle complex problems** – different models capture different patterns.
6. **Bias–variance trade-off:** Error = Bias² + Variance + Noise; ensembles lower one or both.

2. Advantages of Ensemble Methods

- Higher **predictive accuracy** (win most ML competitions, e.g., Kaggle).
- **Less overfitting** (bagging/random forest), less underfitting (boosting).
- Works for both classification and regression; handles **imbalanced / high-dimensional** data well.
- Gives feature importance (random forest) and confidence via vote share.

3. Limitations of Ensemble Learning

- **More computation & memory** – many models to train and store.
- **Less interpretable** ("black box") than a single decision tree.
- **Slower prediction** in real-time systems.
- Gains are small if base models are **highly correlated** (make same errors).
- Boosting can **overfit noisy data**; more hyper-parameters to tune.

4. Two Ensemble Techniques

1. **Bagging (Bootstrap Aggregating):**
   - Create several **bootstrap samples** (random sampling with replacement).
   - Train one model (usually a decision tree) on each, **in parallel**.
   - Combine by **majority vote / average** → **reduces variance**. Example: **Random Forest**.
2. **Boosting:**

- Train models **sequentially**; each new model focuses on the **errors** of previous ones (increases weights of misclassified samples).
- Combine by **weighted vote** → **reduces bias**. Examples: **AdaBoost, Gradient Boosting, XGBoost**.

- *(Others: Stacking – meta-learner on base predictions; Voting – combine different algorithms.)*

Example

Predicting loan default: a single decision tree gives 80 % accuracy; a random forest of 100 trees gives 88 % because individual tree errors average out.

Conclusion

Ensemble learning trades extra computation for **higher accuracy and stability** by combining many models; bagging fights variance and boosting fights bias.

Memory trick: "**Bagging = Parallel, ↓Variance; Boosting = Sequential, ↓Bias**" (B-P-V, B-S-B). Limitations = "**CIS**": Cost, Interpretability, Speed.

U3-2. Explain the voting ensemble and the different types of voting mechanisms.

Importance: ★★★ | **Max asked:** 9 M → **write for 11 M** | **Asked in:** Nov-Dec 2023, Nov-Dec 2024, Nov-Dec 2025

Source: ML Unit 3-6.pdf, pp. 72–74 (Sec. 5.2 Voting Ensemble, 5.2.1 Types of Voting: max voting, averaging, weighted averaging; book pp. 5-5 to 5-7).

Asked as: *Explain different types of voting mechanisms in ensemble learning.* (Nov-Dec 2023 Q5(b), 6 M) | *Explain any three voting mechanism in ensemble learning.* (Nov-Dec 2024 Q6(b), 9 M) | *Describe the concept of a voting ensemble. What are the different types of voting techniques?* (Nov-Dec 2025 Q6(b), 6 M)

Definition

A **voting ensemble** combines predictions of **several different models** (usually heterogeneous – e.g., logistic regression, decision tree, SVM) trained on the same data; the final output is decided by **voting** (classification) or **averaging** (regression).

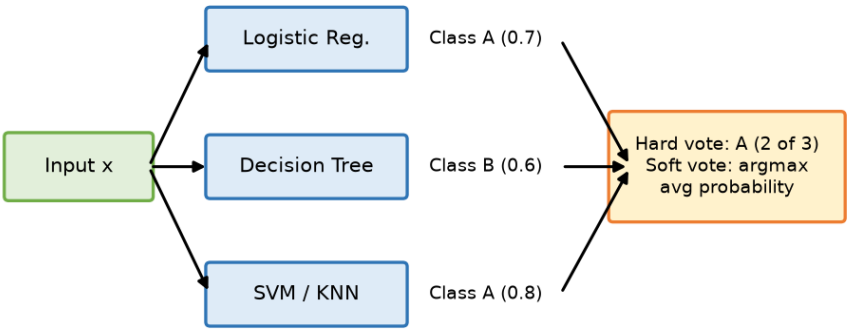


Fig. Voting ensemble: three classifiers vote for the final class

- **Input x:** given to every base classifier.
- **Base classifiers:** each predicts a class (and probability).
- **Voting block:** hard vote picks the majority label; soft vote picks the class with highest average probability.

1. Max Voting (Hard / Majority Voting)

- Used for **classification**. Each model gives **one class label**; class with **most votes** wins.

$$\hat{y} = \text{mode}\{ h_1(x), h_2(x), \ldots, h_m(x) \}$$

- **Example:** Model1 = A, Model2 = B, Model3 = A → **A** (2 of 3).

- Variants: **plurality voting** (most votes, even < 50 %) and **absolute majority** (> 50 % needed, else no decision).

2. Soft Voting

- Each model gives **class probabilities**; average them and pick the **highest**.

$$\hat{y} = \text{argmax}_k \, (1/m) \, \Sigma_i \, P_i(k \mid x)$$

- **Example:** P(A) = 0.7, 0.4, 0.8 → avg 0.63; P(B) = 0.3, 0.6, 0.2 → avg 0.37 → **A**.
- Usually **better** than hard voting when models are well-calibrated.
- Your textbook groups voting types as **Max Voting, Averaging, Weighted Averaging**; soft voting is averaging of class probabilities.

3. Weighted Averaging (Weighted Voting)

- Better models get **higher weights** w_i (based on validation accuracy).

$$\hat{y} = \text{argmax}_k \, \Sigma_i \, w_i \cdot [h_i(x) = k] \quad \text{(or weighted average of probabilities)}$$

- **Example:** weights 0.5, 0.2, 0.3 with votes A, B, B → A = 0.5, B = 0.5 → tie broken by highest-weight model.

4. Averaging (for regression / probabilities)

- **Simple average** $\hat{y} = (1/m) \, \Sigma \, \hat{y}_i$, or **weighted average** $\Sigma \, w_i \hat{y}_i$.
- **Example:** house price predictions 50, 52, 54 L → **52 L**.

Type	Uses	Rule
Max voting (hard)	Labels	Most votes
Plurality	Labels	Largest count (no 50 % needed)
Soft	Probabilities	Highest average probability
Weighted averaging	Labels / probs + weights	Highest weighted score
Averaging	Numeric outputs	Mean / weighted mean

5. Advantages

- Simple; improves accuracy if base models are **accurate and diverse**; reduces risk of picking one bad model.

Conclusion

Voting is the simplest ensemble – **hard** voting counts labels, **soft** voting averages probabilities, **weighted** voting trusts better models more.

Memory trick – "HSW-A": **H**ard (heads count), **S**oft (sum of probabilities), **W**eighted (wise get more), **A**verage (for numbers).

U3-3. Explain the stacking approach of ensemble learning with example. What methods are used for variance reduction in stacking?

Importance: ★★★ | **Max asked:** 9 M → **write for 11 M** | **Asked in:** Nov-Dec 2024, Nov-Dec 2025

Source: ML Unit 3-6.pdf, pp. 79–80 (stacking frame, meta-learner, cross-validation; variance reduction; book pp. 5-12, 5-13).

Asked as: *Elaborate stacking approach of ensemble with example.* (Nov-Dec 2024 Q5(b), 9 M) | *Discuss the concept of stacking in ensemble learning. What are the different methods used for variance reduction in stacking?* (Nov-Dec 2025 Q5(b), 6 M)

Definition

Stacking (Stacked Generalisation) is a **heterogeneous ensemble** in which several **base models (level-0)** are trained on the data and their predictions are used as **input features for a meta-model (level-1, meta-learner)** that learns how to **best combine** them.

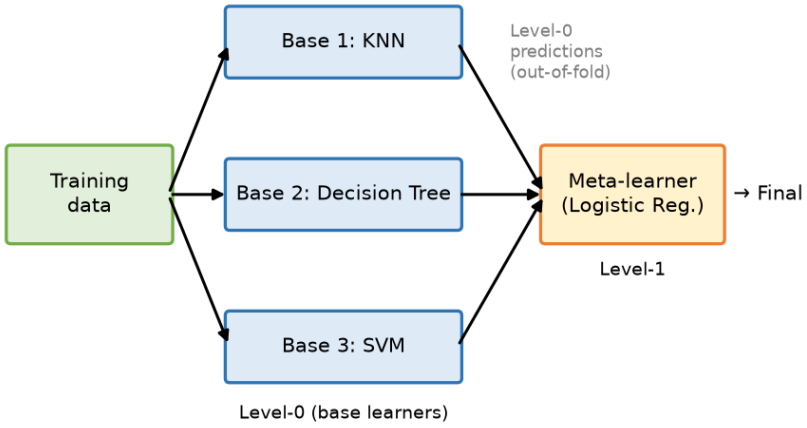


Fig. Stacking: base learners' predictions feed a meta-learner

- **Level-0 base learners:** different algorithms (KNN, decision tree, SVM) trained on the training set.
- **Level-0 predictions:** produced on held-out (out-of-fold) data to avoid leakage.
- **Meta-learner (Level-1):** a simple model (often logistic/linear regression) trained on these predictions to give the final output.

1. Steps of Stacking

1. Split the training data into **K folds**.
2. For each base model, train on K−1 folds and **predict on the held-out fold** (out-of-fold predictions); repeat for all folds.
3. Collect out-of-fold predictions of all base models → **new feature matrix** (one column per model).
4. Train the **meta-learner** on this matrix with the original targets.
5. Retrain each base model on the **full** training data.
6. **Prediction:** base models predict on test data → their outputs go to the meta-learner → **final prediction**.

2. Example

Predict whether a customer will buy (Yes/No):

- Base models: **KNN → 0.62, Decision Tree → 0.80, SVM → 0.55** (probability of Yes).
- Meta-learner (logistic regression) learned weights, e.g., 0.2, 0.6, 0.2 → combined score ≈ 0.70 → **Yes**.
- The meta-learner learns that the tree is most reliable for this data.

3. Variance Reduction Methods in Stacking

1. **Averaging over independent models** (textbook): if base models are trained on independent data, averaging them **reduces variance without increasing bias** (the bagging idea).
2. **K-fold cross-validated (out-of-fold) predictions** – meta-learner never sees predictions on data the base model was trained on → reduces overfitting/variance.
3. **Diverse base learners** – different algorithms make **uncorrelated errors**, so combining reduces variance.
4. **Simple / regularised meta-learner** – logistic/ridge regression (L1/L2) prevents the level-1 model from overfitting.
5. **Bagging the base models** (e.g., use random forest as a base learner) or **averaging predictions across folds** for test data.
6. **Blending** – use a separate hold-out set for level-1 training; use **probabilities** instead of hard labels (smoother features).

4. Advantages / Disadvantages

- **+** Usually more accurate than any single base model or simple voting; uses strengths of different algorithms.
- **−** Complex, computationally expensive, harder to interpret; risk of data leakage if CV is not used.

Conclusion

Stacking learns **how to combine** models: diverse base learners make predictions and a meta-learner, trained on out-of-fold predictions, gives the final output.

Memory trick: "**Stack = Students + Teacher**" – students (base models) answer, the teacher (meta-learner) decides whom to trust. Variance reduction = "**CV, Diversity, Simple meta**".

U3-4. Explain the Adaptive Boosting (AdaBoost) algorithm in detail.

Importance: ★★ | **Max asked:** 6 M → **write for 8 M** | **Asked in:** Nov-Dec 2023, Nov-Dec 2025

Source: ML Unit 3-6.pdf, pp. 77–80 (boosting, AdaBoost, AdaBoost stump; book pp. 5-10 to 5-13). Weight-update formulas are the standard ones (the OCR of the formula lines was unreadable).

Asked as: *Explain AdaBoost algorithm in detail.* (Nov-Dec 2023 Q5(c), 6 M) | *Explain the Adaptive Boosting (AdaBoost) algorithm in detail.* (Nov-Dec 2025 Q6(c), 6 M)

Definition

AdaBoost (Adaptive Boosting), proposed by Freund & Schapire, is a **sequential boosting** algorithm that combines many **weak learners** (usually **decision stumps** – one-level trees) into a **strong classifier**. After each round it **increases the weights of misclassified samples** so the next learner focuses on hard examples.

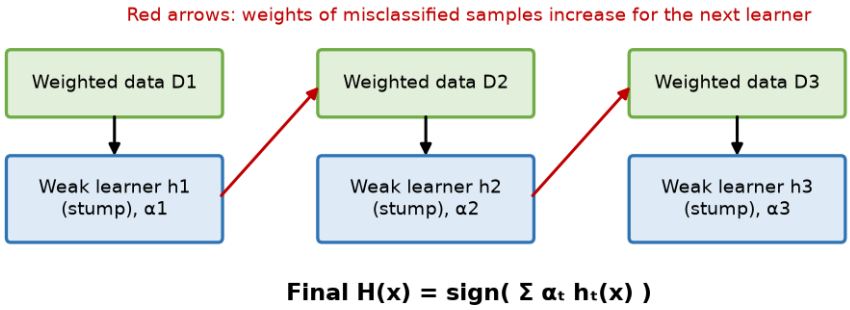


Fig. AdaBoost: re-weighted data and weak learners combined by weighted vote

- **Weighted data D_1, D_2, D_3:** same samples, weights updated each round.
- **Weak learner h_t:** stump trained on weighted data; gets a "say" α_t.
- **Red arrows:** misclassified samples get higher weights.
- **Final H(x):** sign of the weighted sum of all weak learners.

1. Algorithm (binary labels $y_i \in \{-1, +1\}$)

1. **Initialise weights** of N samples equally: $w_i = 1/N$.
2. For t = 1 to T:
   - **(a)** Train weak learner $\mathbf{h_t}$ on data weighted by w_i.
   - **(b)** Compute **weighted error**:

$$\varepsilon_t = \Sigma\, w_i \cdot [h_t(x_i) \neq y_i]$$

   - **(c)** Compute **learner weight (amount of say)**:

$$\alpha_t = \tfrac{1}{2} \ln(\,(1 - \varepsilon_t) / \varepsilon_t\,)$$

   - **(d) Update sample weights:**

$$w_i \leftarrow w_i \cdot \exp(\,-\alpha_t\, y_i\, h_t(x_i)\,)$$

- Misclassified → weight **increases**; correct → weight **decreases**.

(e) Normalise weights so $\Sigma w_i = 1$.

3. Final strong classifier:

$$H(x) = \text{sign}(\Sigma_t \alpha_t h_t(x))$$

2. Example

10 samples, each weight 0.1. Stump-1 misclassifies 3 → $\varepsilon_1 = 0.3$, $\alpha_1 = ½ \ln(0.7/0.3) =$ **0.42**. The 3 wrong samples' weights rise (0.1 × e^0.42 ≈ 0.152) and the 7 correct ones fall (0.1 × e^−0.42 ≈ 0.066); after normalising, wrong ones ≈ 0.167, correct ones ≈ 0.071. Stump-2 therefore concentrates on those 3 samples.

3. Advantages / Disadvantages

- **+** Simple, few parameters (T); high accuracy from weak learners; reduces **bias**; less prone to overfitting on clean data; gives feature importance.
- **− Sensitive to noise and outliers** (they get huge weights); sequential → cannot parallelise; weak learner must be better than random ($\varepsilon < 0.5$).

4. Applications

- **Face detection (Viola–Jones)**, spam filtering, customer churn, medical diagnosis.

Conclusion

AdaBoost adaptively **re-weights samples** and **weights learners by accuracy**, turning many weak stumps into one strong classifier.

Memory trick – "W-E-A-U-N": Weights equal → **E**rror → **A**lpha → **U**pdate → **N**ormalise. Common mistake: forgetting the ½ and ln in α_t.

U3-5. Explain the Random Forest ensemble method with an example.

Importance: ★★ | **Max asked:** 6 M → **write for 8 M** | **Asked in:** Nov-Dec 2023, Nov-Dec 2025

Source: ML Unit 3-6.pdf, pp. 81–85 (Sec. 5.4 Random Forest Ensemble, advantages; book pp. 5-14 to 5-18).

Asked as: *Explain random forest ensembles with an example.* (Nov-Dec 2023 Q6(c), 6 M) | *Describe the Random Forest ensemble method in detail.* (Nov-Dec 2025 Q5(c), 6 M)

Definition

Random Forest is a **bagging-based homogeneous ensemble** of many **decision trees**, where each tree is trained on a **bootstrap sample** and, at each split, considers only a **random subset of features**. The final output is the **majority vote** (classification) or **average** (regression) of all trees.

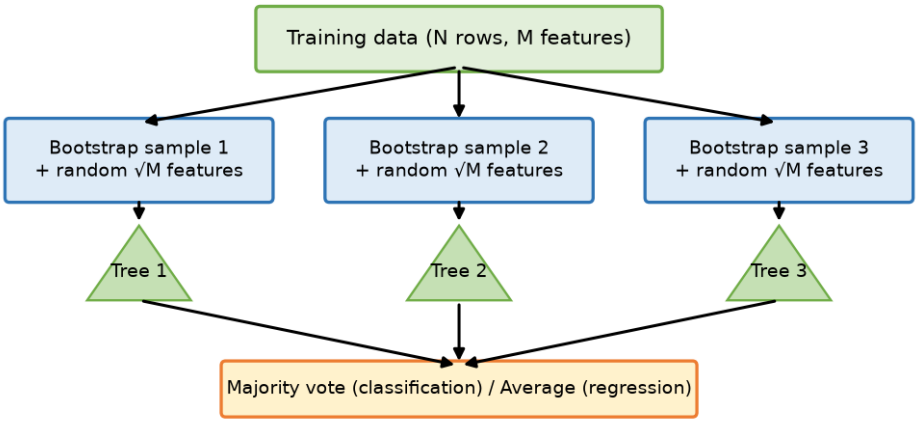


Fig. Random Forest: bootstrap samples + random features → many trees → vote

- **Bootstrap sample:** N rows drawn **with replacement** for each tree (≈ 63 % unique rows).
- **Random features:** at each node only m ≈ √M features (classification) are tried → trees become **decorrelated**.
- **Trees 1…n:** fully grown, unpruned decision trees.
- **Vote / average:** combines tree outputs.

1. Algorithm

1. Choose number of trees **n_estimators** (e.g., 100) and features per split **m** (√M or M/3).
2. For each tree: draw a **bootstrap sample** of size N from the training data.
3. Grow a decision tree; at each node select **m random features** and choose the best split (Gini / entropy) among them.
4. Grow the tree to maximum depth (no pruning).
5. **Predict:** pass a new sample through all trees → **majority vote** / **mean**.
6. *(Out-of-bag (OOB) samples – rows not in a tree's bootstrap – estimate test error without a separate validation set.)*

2. Example

Predict "Play Tennis?" with features Outlook, Temperature, Humidity, Wind; 3 trees:

- Tree 1 (Outlook, Humidity) → **Yes**; Tree 2 (Wind, Temp) → **No**; Tree 3 (Outlook, Wind) → **Yes**.
- Majority vote → **Yes (2 of 3)**.

3. Advantages

- High accuracy; **reduces overfitting/variance** of single trees.
- Handles **missing values, large and high-dimensional data**; works for classification & regression.
- Gives **feature importance**; OOB error estimate; trees train **in parallel**.

4. Disadvantages

- Many trees → **slow prediction**, large memory; less interpretable than one tree.
- Can be biased toward features with many levels; not good at extrapolation in regression.

5. Applications

- Banking (loan default, fraud), medicine (disease prediction), e-commerce recommendation, remote sensing.

Conclusion

Random Forest = **bagging + random feature selection + decision trees**; the randomness makes trees diverse, and voting gives an accurate, robust model.

Memory trick: "**Random rows + Random columns = Random Forest**." Two randomness sources: bootstrap rows and feature subsets.

U3-6. Differentiate between Homogeneous and Heterogeneous ensemble methods, with examples.

Importance: ★★ | **Max asked:** 6 M → **write for 8 M** | **Asked in:** Nov-Dec 2023, Nov-Dec 2025

Source: ML Unit 3-6.pdf, p. 71 (homogeneous and heterogeneous ensemble methods; book p. 5-4).

Asked as: *Compare Homogeneous and Heterogeneous ensemble methods.* (Nov-Dec 2023 Q6(a), 6 M) | *Differentiate between homogeneous and heterogeneous ensemble methods. provide examples of each type.* (Nov-Dec 2025 Q6(a), 6 M)

Definition

In a **homogeneous ensemble** all base learners use the **same algorithm** (diversity comes from different data samples/weights); in a **heterogeneous ensemble** base learners use **different algorithms** (diversity comes from different learning methods).



Fig. Homogeneous (same algorithm) vs heterogeneous (different algorithms)

Basis	Homogeneous ensemble	Heterogeneous ensemble
Base learners	**Same type** (e.g., all decision trees)	**Different types** (KNN, SVM, DT, NB)
Source of diversity	Different **data subsets / sample weights / features**	Different **algorithms (inductive biases)**
Training data	Bootstrap samples or re-weighted data	Usually the **same full dataset**
Combination	Voting / averaging / weighted sum	Voting or a **meta-learner**
Typical methods	**Bagging, Random Forest, AdaBoost, Gradient Boosting**	**Voting classifier, Stacking, Blending**
Implementation	Easier, one algorithm to tune	More complex, many algorithms to tune
Error correlation	Higher (similar models)	Lower (different models)
Example	Random Forest of 100 trees for loan approval	LR + SVM + KNN voting for spam detection

Conclusion

Homogeneous ensembles get diversity from **data**, heterogeneous ensembles from **algorithms**; both aim to combine diverse models for better accuracy.

Memory trick: Homo = same (homogeneous milk – one type); **Hetero = different**. Homo → Bagging/Boosting; Hetero → Voting/Stacking.

U3-7. Explain Gradient Boosting ensemble learning technique.

Importance: ★ | **Max asked:** 9 M → **write for 11 M** | **Asked in:** Nov-Dec 2024

Source: ⚠ Not found in your materials. Standard textbook answer. (Gradient Boosting is only listed in the syllabus/contents on p. 68 of ML Unit 3-6.pdf; no explanation was found in the text.)

Asked as: *Describe ensemble learning. Explain Gradient Boosting ensemble learning techniques.* (Nov-Dec 2024 Q6(a), 9 M)

Definition

Ensemble learning combines multiple models for better accuracy. **Gradient Boosting** is a **sequential boosting** technique in which each new weak learner (a small regression tree) is trained to predict the **residual errors (negative gradients of the loss)** of the current ensemble, and is added with a **learning rate η**. It performs **gradient descent in function space**.

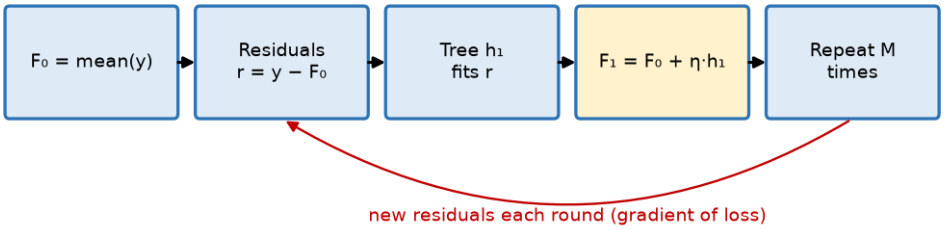


Fig. Gradient Boosting: start with mean, fit trees to residuals, add with learning rate

- **F_0:** initial prediction (mean of y for squared loss).
- **Residuals:** errors of the current model = negative gradient.
- **Tree h_m:** fitted to residuals. **Update:** $F_m = F_{m-1} + \eta \cdot h_m$. Repeat M times.

1. Algorithm (regression, squared loss $L = \frac{1}{2}(y - F)^2$)

1. **Initialise** $F_0(x)$ = mean(y).
2. For m = 1 to M:
   (a) Compute **pseudo-residuals**:

$$r_i = -\partial L(y_i, F(x_i)) / \partial F(x_i) = y_i - F_{m-1}(x_i)$$

   (b) Fit a weak learner (shallow tree) $\mathbf{h_m(x)}$ to the residuals r_i.
   (c) Update the model:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

   - η = learning rate (0.01–0.3), controls step size.
3. Output final model **F_M(x)**.

2. Example

Predict salary (in ₹ L) for 3 people: actual = 10, 14, 18.

1. F_0 = mean = **14**; residuals = −4, 0, +4.
2. Tree h_1 learns residuals ≈ −4, 0, +4; with η = 0.5: F_1 = 14 + 0.5 × (−4, 0, 4) = **12, 14, 16**.
3. New residuals = −2, 0, +2 → tree h_2 → F_2 = **11, 14, 17** … predictions move closer to 10, 14, 18 each round.

3. Key Hyper-parameters

- **n_estimators (M)**, **learning rate η**, **max_depth** of trees (3–8), **subsample** (stochastic GB).
- Small η + more trees = better generalisation.

4. Advantages / Disadvantages

- **+** Very high accuracy; works with **any differentiable loss** (regression, classification, ranking); handles mixed data; feature importance.
- **−** Sequential → **slow training**; **overfits** if too many trees/large η; many hyper-parameters; sensitive to outliers (with squared loss).
- Popular implementations: **XGBoost, LightGBM, CatBoost**.

5. AdaBoost vs Gradient Boosting (quick)

AdaBoost	Gradient Boosting
Re-weights **samples**	Fits **residuals (gradients)**
Exponential loss	Any differentiable loss
Learner weight α_t	Learning rate η

Conclusion

Gradient Boosting builds the model **step by step**, each tree correcting the residual errors of the previous ensemble, giving state-of-the-art accuracy on tabular data.

Memory trick: "**Mean → Residual → Tree → Add (×η) → Repeat**" (MR-TAR). GB = "gradient descent with trees".

U3-8. Differentiate the Bagging and Boosting approaches of ensemble learning.

Importance: ★ | **Max asked:** 6 M → **write for 8 M** | **Asked in:** Nov-Dec 2023

Source: ML Unit 3-6.pdf, pp. 75–77 (bagging: bootstrapping and aggregation; boosting; book pp. 5-8 to 5-10). Comparison table is standard.

Asked as: *Differentiate the Bagging and Boosting approach of ensemble learning.* (Nov-Dec 2023 Q5(a), 6 M)

Definition

Bagging (Bootstrap Aggregating) trains models **independently in parallel** on bootstrap samples and averages them; **Boosting** trains models **sequentially**, each focusing on the errors of the previous ones, and combines them by weighted vote.

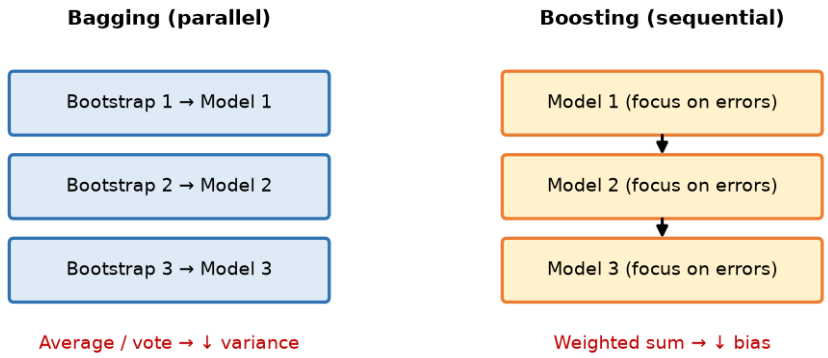


Fig. Bagging (parallel, independent) vs Boosting (sequential, error-focused)

Basis	Bagging	Boosting
Full form / idea	**Bootstrap Aggregating** – average many models	Convert **weak learners into a strong** learner
Training order	**Parallel**, independent	**Sequential**, each depends on previous
Data sampling	Random **bootstrap samples with replacement**	Same data, **re-weighted** (misclassified ↑) or residuals
Model weights	**Equal** weight to every model	**Weighted** by performance (α_t)
Main goal	**Reduce variance** (overfitting)	**Reduce bias** (underfitting) (and variance)
Base learner	Strong, complex learners (deep trees)	Weak learners (**stumps**, shallow trees)
Combination	Majority vote / simple average	Weighted vote / weighted sum
Noise / outliers	**Robust**	**Sensitive** (outliers get high weight)
Overfitting risk	Low	Higher if too many rounds
Speed	Fast (parallelisable)	Slower (sequential)
Examples	**Random Forest**, Bagged trees	**AdaBoost, Gradient Boosting, XGBoost**

Conclusion

Use **bagging** when a model overfits (high variance) and **boosting** when it underfits (high bias); both are homogeneous ensembles.

Memory trick: "Bagging = Bag of independent votes; Boosting = Boost the weak one by one."

4. Last-Minute Revision Sheet

Key Definitions

- **Ensemble:** combine many base learners → better accuracy & stability ("wisdom of crowd").
- **Bagging:** bootstrap samples, parallel, equal vote → ↓ variance (Random Forest).
- **Boosting:** sequential, focus on errors, weighted vote → ↓ bias (AdaBoost, GBM, XGBoost).
- **Stacking:** base learners (level-0) + meta-learner (level-1) on out-of-fold predictions.
- **Voting:** hard (labels), soft (probabilities), weighted, averaging.
- **Homogeneous** = same algorithm; **heterogeneous** = different algorithms.
- **Random Forest** = bagging + random feature subset at each split.

Formulas

Item	Formula
Hard vote	$\hat{y} = \text{mode}\{h_1(x), \ldots, h_m(x)\}$
Soft vote	$\hat{y} = \text{argmax}_k\ (1/m)\Sigma P_i(k\|x)$
AdaBoost error	$\varepsilon_t = \Sigma w_i[h_t(x_i) \neq y_i]$
AdaBoost learner weight	$\alpha_t = ½\ \ln((1 - \varepsilon_t)/\varepsilon_t)$
AdaBoost weight update	$w_i \leftarrow w_i \cdot \exp(-\alpha_t y_i h_t(x_i))$, then normalise
AdaBoost final	$H(x) = \text{sign}(\Sigma \alpha_t h_t(x))$
Gradient boosting	$r_i = y_i - F_{m-1}(x_i)$; $F_m = F_{m-1} + \eta \cdot h_m$
Bias–variance	Error = Bias² + Variance + Noise

Memory Tricks

- **B-P-V / B-S-B:** Bagging-Parallel-Variance; Boosting-Sequential-Bias.
- Voting = **HSW-A**; AdaBoost = **W-E-A-U-N**; GB = **MR-TAR**.
- Stacking = **Students + Teacher**; RF = **random rows + random columns**.
- Ensemble limitations = **CIS** (Cost, Interpretability, Speed).

5. Assumptions & Gaps

- **Unit mapping:** Q5/Q6 of each paper are taken as Unit 3 (= syllabus Unit V, Ensemble Learning).
- **2023 Q6(b)** "What is ensemble learning? Explain any two techniques" is merged with the 2024/2025 concept/need questions (U3-1); bagging and boosting are given as the two techniques.
- **2024 Q6(a)** asks to "describe ensemble learning" plus Gradient Boosting; the ensemble definition is in U3-1 and repeated briefly in U3-7.
- ⚠ **Not found:** U3-7 Gradient Boosting is only listed in the syllabus (p. 68); the answer is a standard textbook answer.